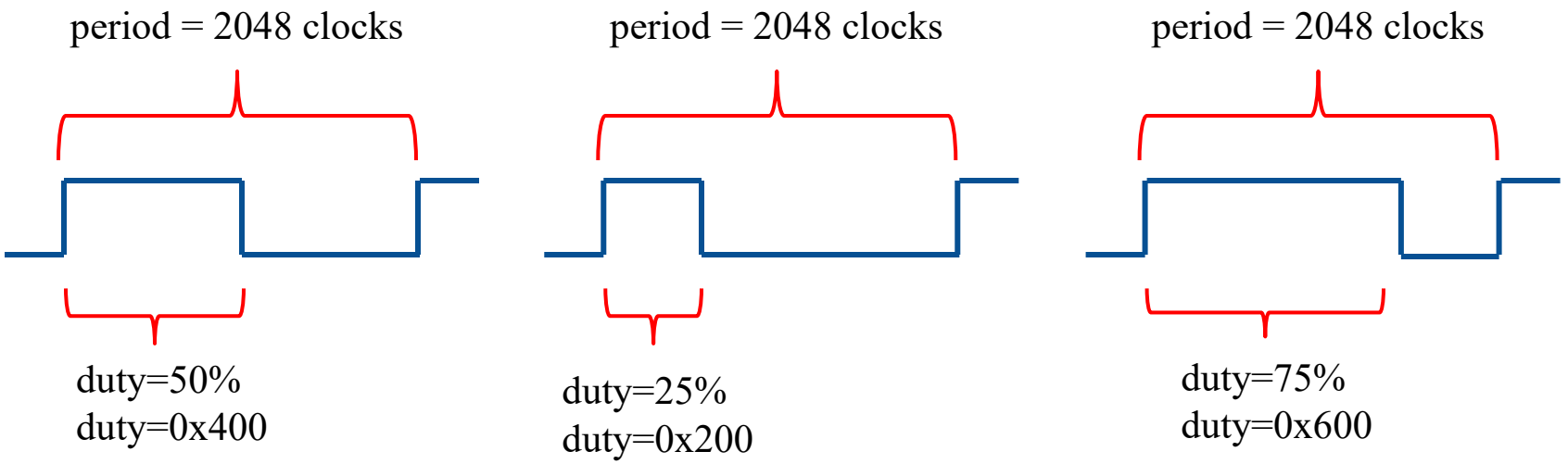
Exercise 09: PWM (**P**ulse **W**idth **M**odulation)



- A PWM signal is used to control the magnitude of something. In our case it will be used to control the magnitude of drive to a DC motor.
- The signal typically will have a fixed frequency, and the duty cycle will determine the magnitude.
- Essentially, we are turning the motor on full bore for the duration of the high pulse of the PWM signal, and off for the remainder of the cycle.
- If the period of the PWM signal is short enough then this on/off control is essentially filtered and looks like a nice analog drive at whatever percentage the duty cycle is.

Exercise 09: PWM (**P**ulse **W**idth **M**odulation)

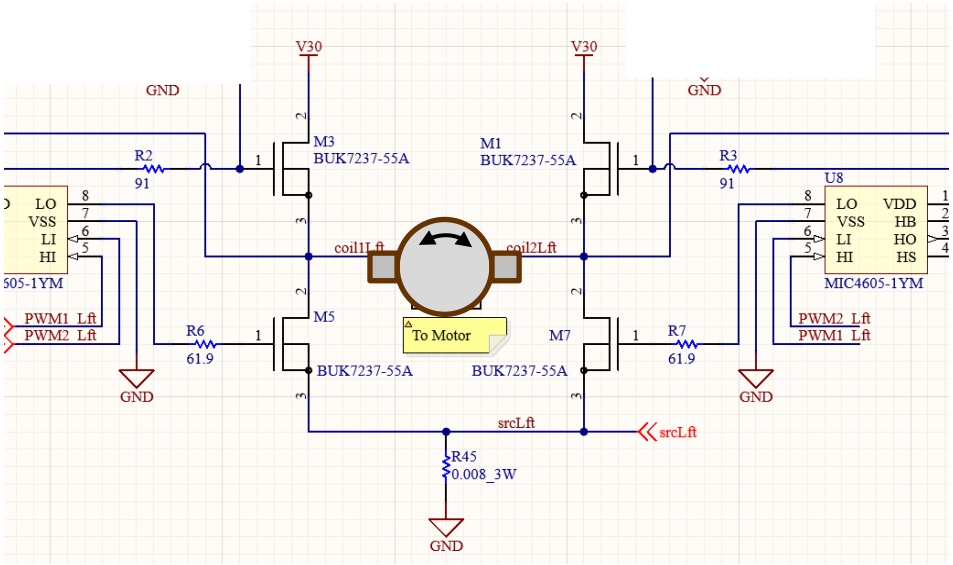
- PWM is better than analog drive because it is more efficient.
  - When the switches to drive the motor are on, they are on hard so have very little resistance and therefore negligible I^2R loss.
  - When they are off, they have no current therefore no I2R loss.
  - If we did an actual analog drive to the motor coils we would have I2R losses over partially on pass elements.
- A frequency in the 20kHz area is appropriate for a DC motor. We have a 50MHz clock on the DE0-Nano, so 11-bit period works pretty nice.

$$\frac{50\text{MHz}}{2^{11}} = 24.4\text{kHz}$$

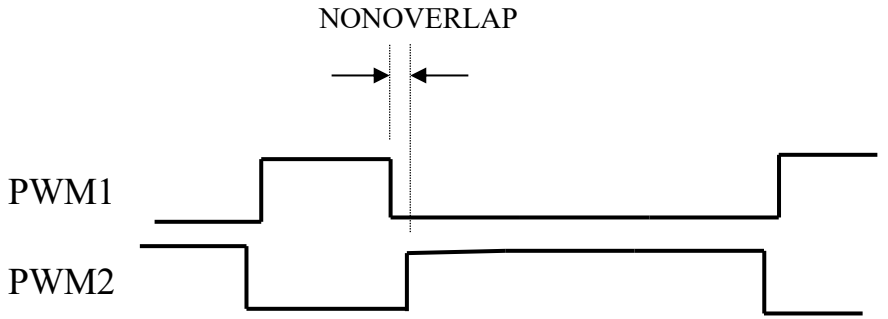
- A PWM module cannot achieve both zero duty cycle and 100% duty cycle. If zero means zero duty, then what does 0x7FF mean? It means we are on for 2047 out of 2048 clocks, so not quite 100%. But we are fine with this.

Need for non-overlap

- The motors can be driven forward or reverse from the 30V battery through the circuit shown.
- The I/Os of our little FPGA cannot drive large motors directly so there are power MOSFETs involved.



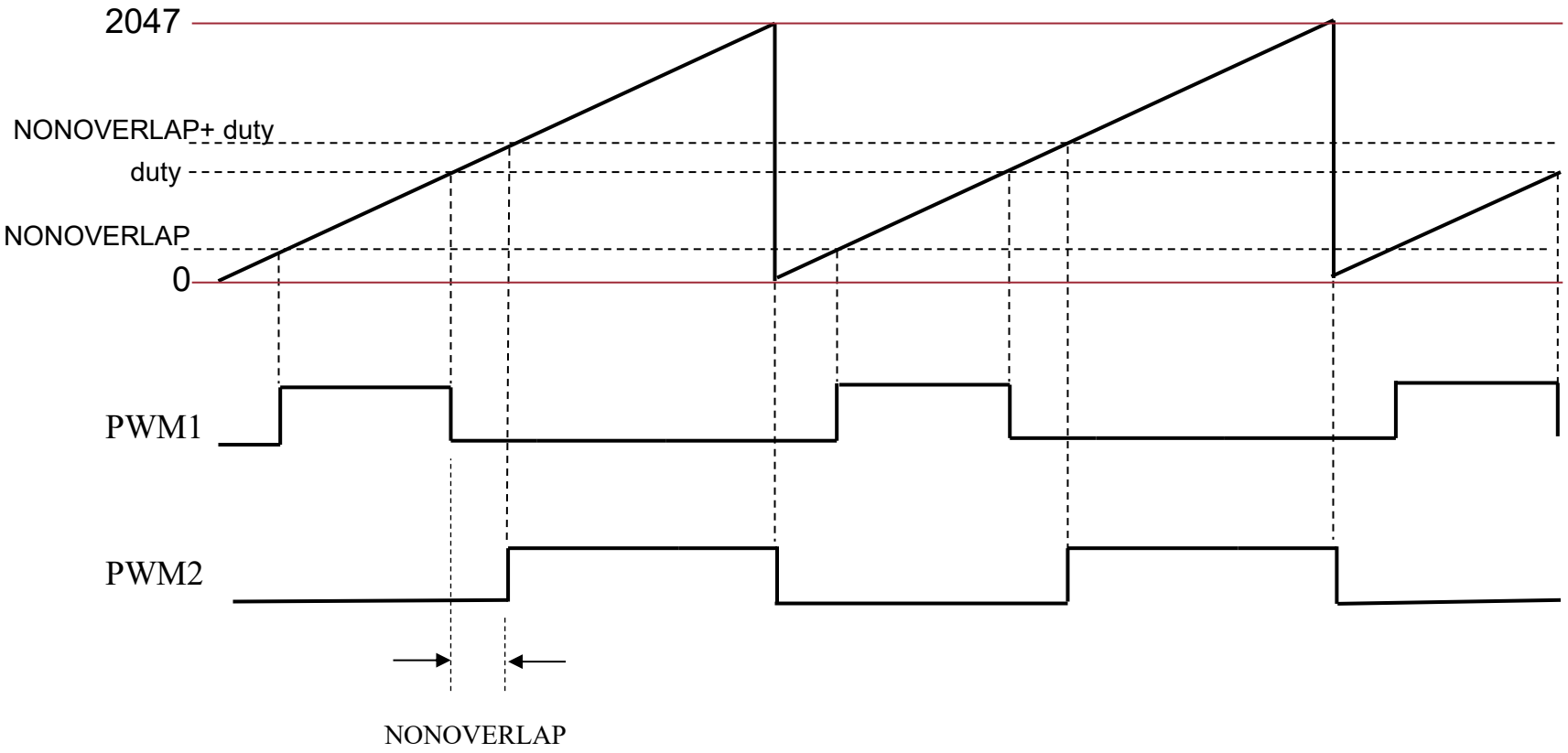
- PWM1 & PWM2 are complementary signals *(opposite of each other)* but they also have a non-overlap time *(time in which they are both low)*



- Why is the non-overlap necessary?
  - If both the power MOSFETs in a stack were on, then current would be shooting through the transistors from 30V supply to ground.
  - We need to ensure both MOSFETs in a stack are never on at same time.

PWM11: Implementing non-overlap

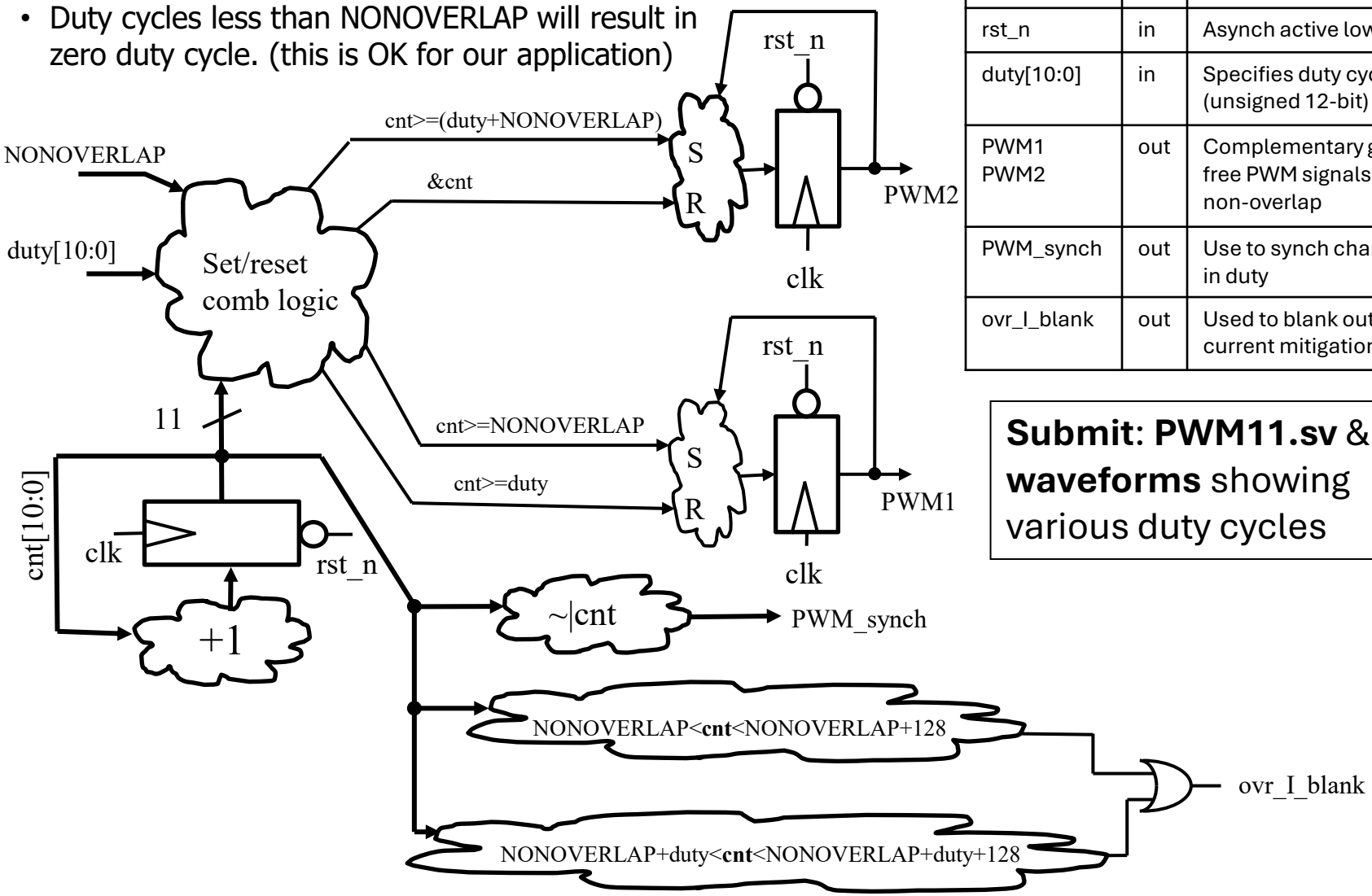
- Think of the 11-bit free-running counter. It ramps from 0 to 2047 then rolls over to 0 again. Think of it plotted as an analog wave.



- PWM1 runs from **NONOVERLAP** to **duty**
- PWM2 runs from **duty+NONOVERLAP** to 2047

PWM11 Implemention

- We will have a **localparam** called NONOVERLAP and set it to 11'h040.
- Duty cycles less than NONOVERLAP will result in zero duty cycle. (this is OK for our application)



Signal	Dir	Description
clk	in	50MHz system clk
rst_n	in	Asynch active low
duty[10:0]	in	Specifies duty cycle (unsigned 12-bit)
PWM1 PWM2	out	Complementary glitch free PWM signals with non-overlap
PWM_synch	out	Use to synch changes in duty
ovr_I_blank	out	Used to blank out over current mitigation

Submit: PWM11.sv & **waveforms** showing various duty cycles

PWM_synch & ovr_I_blank

- The Duty cycle applied to the PWM is generated from another unit. However, changes in the PWM duty cycle should be synched with the PWM cycle so changes to duty do not occur in the middle of a PWM period.
- We need to generate a **PWM_synch** signal to allow the unit driving duty to synchronize changes in duty to the PWM cycle. This signal can simply be when the 11-bit counter is all zeros.

- We have an over current detect circuit that will shut down motor drive if instantaneous current in the motor is too high. Over current detect circuits have a tendancy to false trigger early in the application of a new PWM pulse to the motor.
- We need to generate a blanking signal, so we ignore the over current detect if we are in the first 128 clock cycles of either **PWM1** or **PWM2**. This signal can be assigned based on range checking **cnt** vs NONOVERLAP and **duty**.

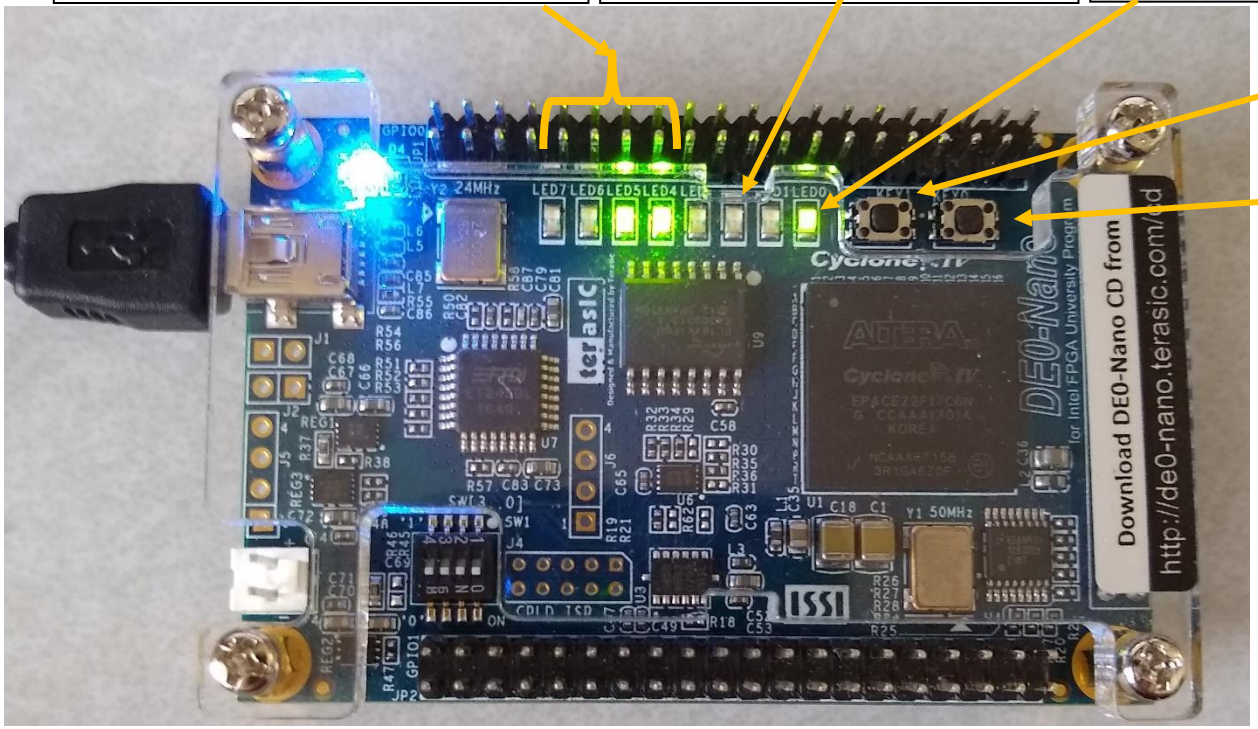
Running PWM11 on DE-0 Nano

- We will use your PWM11 block to modulate the intensity of the LED lights on the FPGA board
- A push button will be used as system reset
- Another push button will be used to incr/decr intensity of LED light
  - We will have a counter to track intensity value

Upper nibble of LEDs will be your 4-bit intensity counter

LED[2] will vary in intensity with duty cycle of PWM

Lowest bit of LEDs will vary in intensity with duty cycle of PWM



"inc" push button

"RST_n" push button

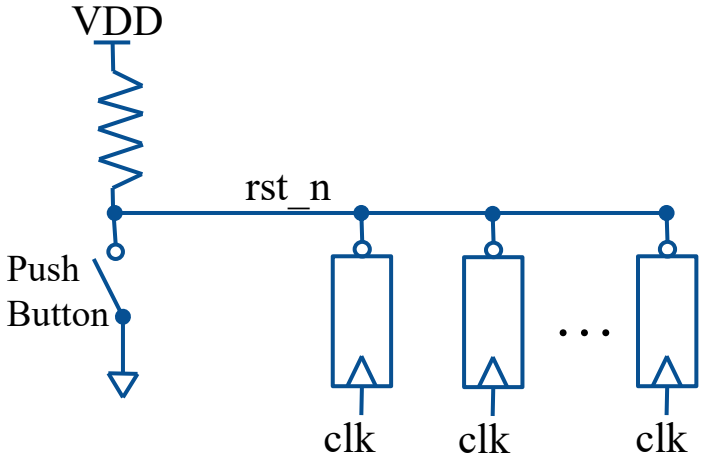
Intensity of LED[0] should increase at first as counter increases.

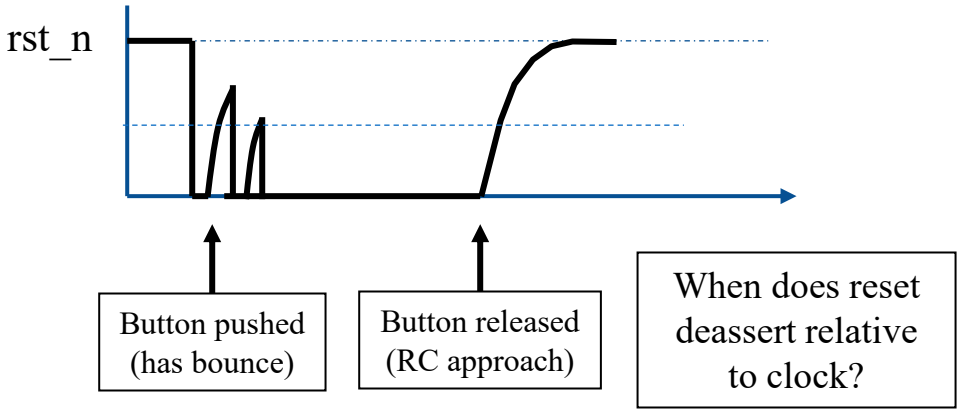
When count gets to 1111 then counter should reverse and start counting down.

Obtaining a Decent rst_n

- On the FPGA board we have a couple of push buttons. We will use one as the source for our asynchronous reset.

- Generic push buttons can be thought of as a momentary switch to ground with a pull-up resistor.



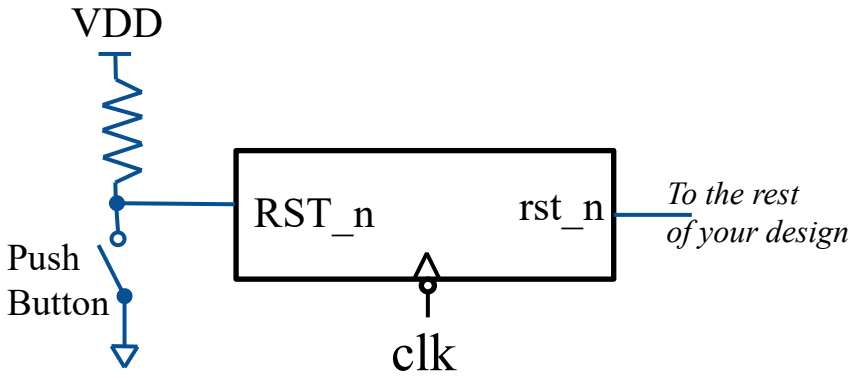


Imagine what a disaster this implementation of a global reset would be!!

- I want you to build a **reset synchronizer** to avoid the above

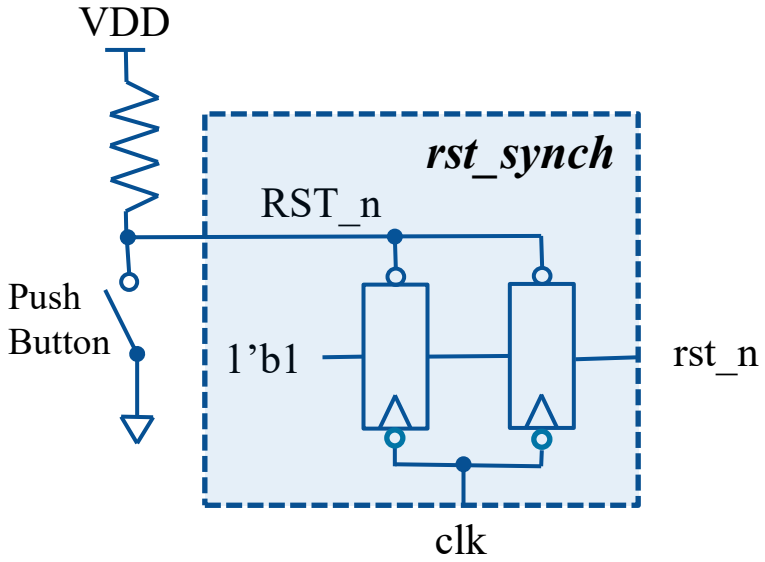
Synchronizing a PB reset

- Remember, you want your reset de-asserted on the **opposite edge** of clock that your other flops are active on. This means we want our reset to de-assert (rise) on negative edge of clock.



It will have an interface of:

- **RST_n** = raw input from push button
- **clk** = clock, and we use negative edge
- **rst_n** = our syncrhronized output which will form the global reset to the rest of our chip.

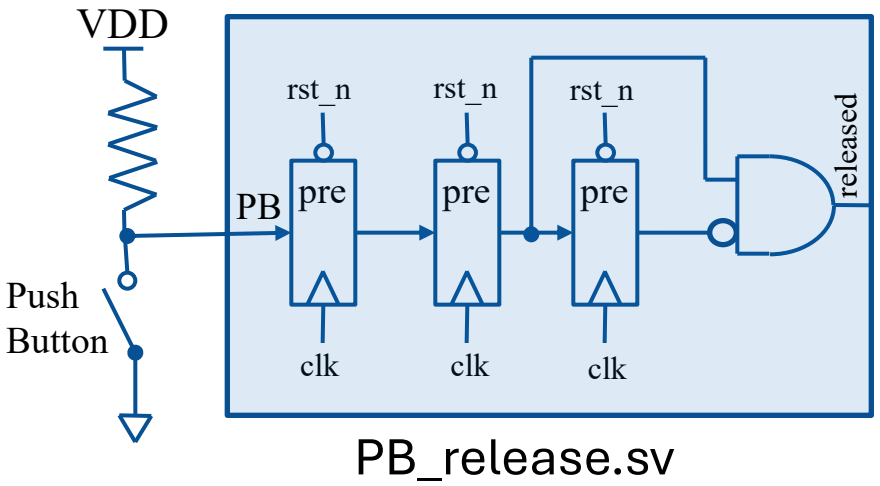


Push of the button will asynch reset the two flops. When button is released we have a double flopping (metastability reasons) to produce our global **rst_n**. The flops are negative edge triggered so our global reset will deassert on the opposite edge of all our other flops.

Code this reset synch unit (**rst_synch.sv**)

Synching & Edge Detecting a PB input

- Incr/decr intensity value is done using a push button as well
- Implement the following circuit and call it PB_release.sv



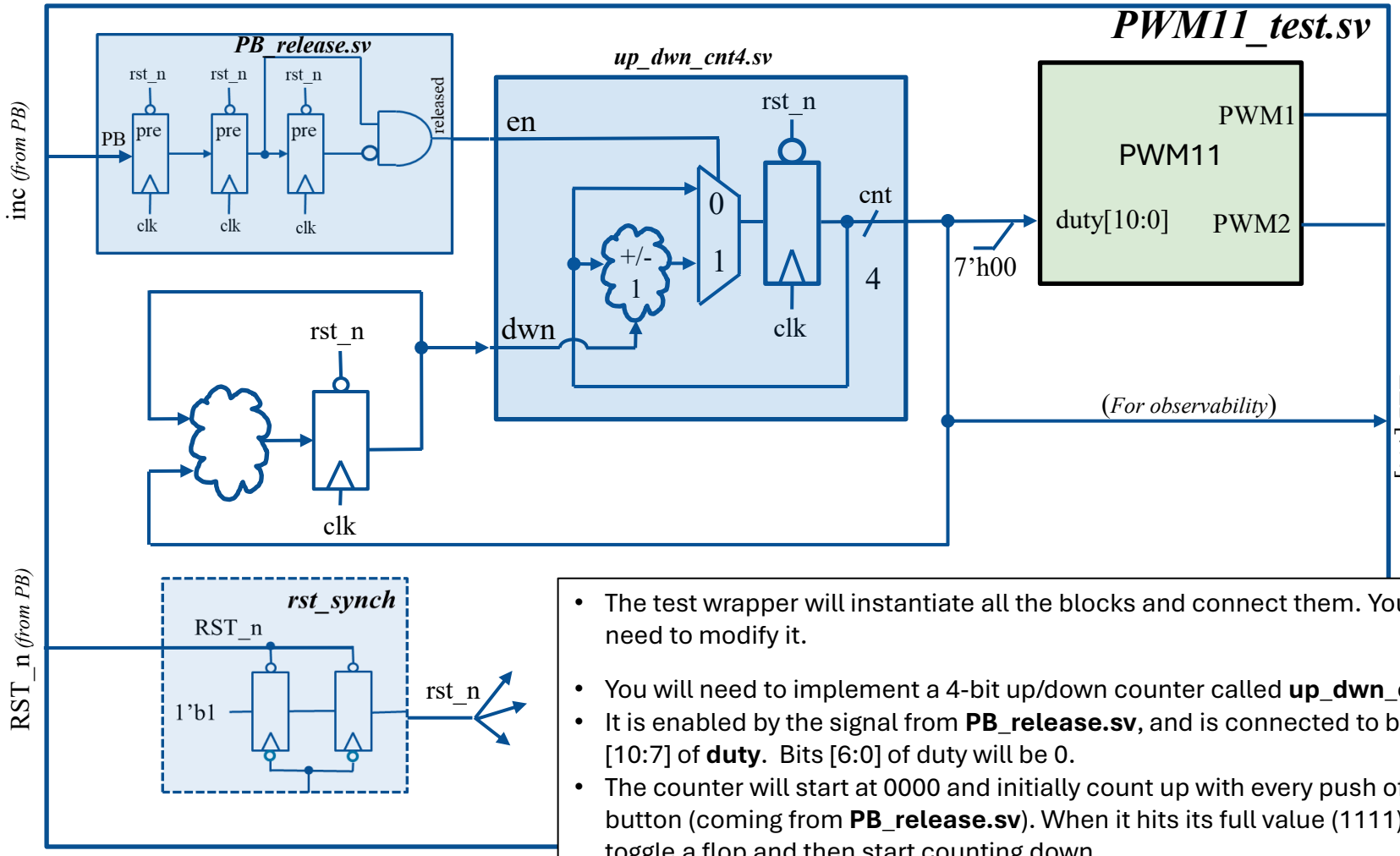
PB_release.sv

Does this look familiar?

- PB is asynch input
- We need to handle metastability
- We want to detect a rising edge

PWM11_test

- **PWM11_test.sv** is provided and is a wrapper that will use your PWM11 to vary the intensity of a LED.



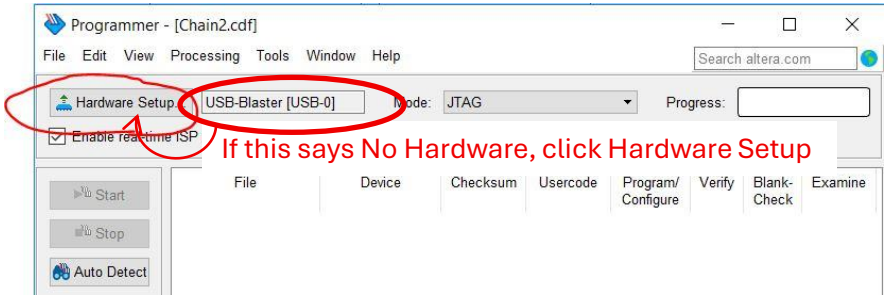
- The test wrapper will instantiate all the blocks and connect them. You do not need to modify it.
- You will need to implement a 4-bit up/down counter called **up_dwn_cnt4.sv**
- It is enabled by the signal from **PB_release.sv**, and is connected to bits [10:7] of **duty**. Bits [6:0] of duty will be 0.
- The counter will start at 0000 and initially count up with every push of a button (coming from **PB_release.sv**). When it hits its full value (1111), it will toggle a flop and then start counting down.

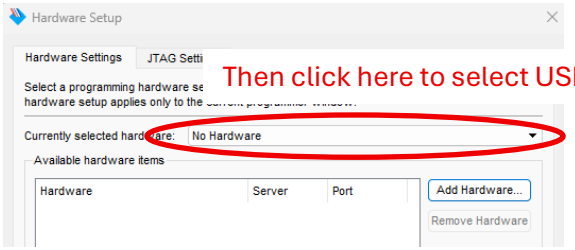
Mapping onto the FPGA

- Create an **Exercise09** directory under your ECE551 area of your I: drive
- Download the **PWM11_test.qpf** (Quartus Project File) and **PWM11_test.qsf**(Quartus Settings File) from the website and store in your Exercise09 directory
- Ensure the following files reside in the directory: rst_synch.sv, PB_release.sv, up_dwn_cnt4.sv, PWM11_test.sv, PWM11.sv

- Open up Quartus & open the project file
  - **File → Open Project** and open up the **PWM11_test.qpf**
  - Make sure the selected device says **Cyclone IV E: EP4CE22F17C6**
  - **Compile** and fix any errors

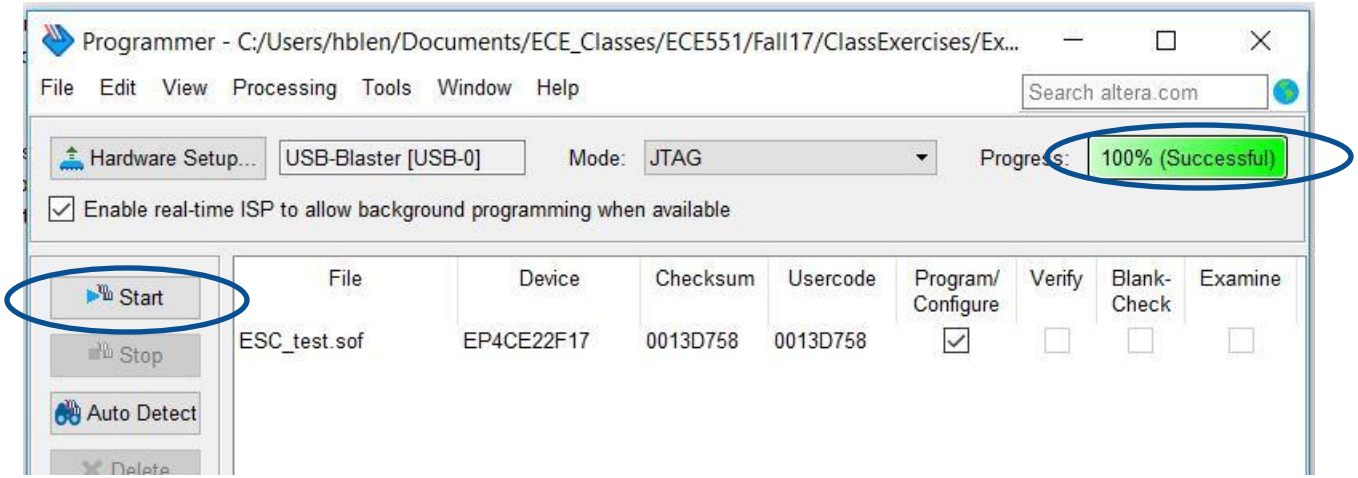
- Demo by Wednesday for Bonus Credit 1%
  - **Tools → Programmer** and check that USB Blaster shows up (it takes a bit on CAE machines…)





Mapping onto the FPGA

- Program the FPGA



- Hit “Start” and look for 100% Success
- Reset the system, and vary the intensity. You should see LED[0] and LED[2] change intensity.

- **Submit**: rst_synch.sv, PB_release.sv, up_dwn_cnt4.sv